

Experiment:1

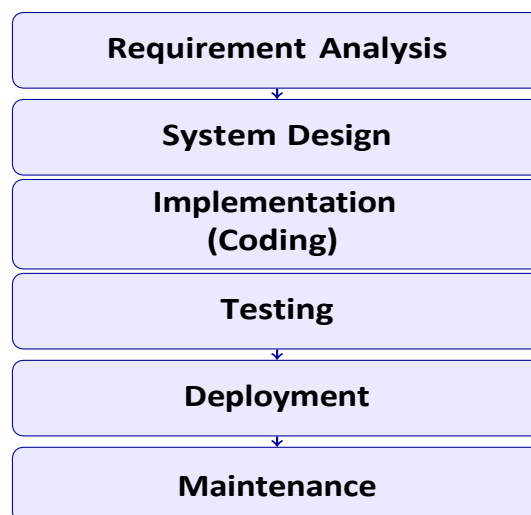
Aim: Introduction to Development Operations, Key concept of Automation and CD/CI, and its relevance to the modern software engineering practices like agile and lean.

1 What Is the Waterfall SDLC Model?

The Waterfall Model is one of the oldest and simplest Software Development Life Cycle (SDLC) models. In this model, the software development process flows step by step in a linear sequence, similar to water flowing down a waterfall.

Each phase must be completed before moving to the next phase, and there is no overlapping between phases. Because of this structured flow, the Waterfall Model is easy to understand, plan, and manage when requirements are clear and stable.

2 Phases of the Waterfall Model



2.1 Requirement Analysis

In this phase, customer needs and project requirements are collected, analyzed, and documented. The goal is to clearly understand what the software must do.

2.2 System Design

The system architecture, database design, user interface design, and technical specifications are planned during this phase.

2.3 Implementation

In the implementation phase, developers write the actual source code based on the approved design documents.

2.4 Testing

The completed software is tested to find and fix defects. Testing usually happens after the coding phase is complete.

2.5 Deployment

After successful testing, the software is delivered and installed in the customer's environment or released to users.

2.6 Maintenance

After deployment, the software is maintained by fixing bugs, improving performance, and making necessary updates.

3 Characteristics of the Waterfall Model

- It follows a linear and sequential development process.
- Each phase must be completed before the next phase begins.
- It involves heavy documentation at every stage.
- It is simple and easy to understand.
- Customer involvement is usually minimal after requirements are finalized.
- Changes are difficult once development has started.

4 Advantages of the Waterfall Model

1. **Simple and easy to understand:** The model is suitable for beginners and small projects because of its clear step-by-step structure.
2. **Easy to manage:** Each phase has clearly defined deliverables, milestones, and review points.
3. **Well documented:** Extensive documentation makes future maintenance and project tracking easier.
4. **Clear project planning:** Cost, timeline, and resources can be estimated more accurately when requirements are stable.
5. **Suitable for stable requirements:** The model works well when customer requirements are clear and unlikely to change.

5 Disadvantages of the Waterfall Model

1. **Difficult to change requirements:** If the client requests changes after development has started, implementing them can be costly and time-consuming.
2. **Testing happens late:** Most testing occurs after coding is completed, so defects may be discovered late in the project.
3. **No working software until the end:** The customer typically cannot use or evaluate the software until the deployment phase.
4. **Higher risk:** Mistakes made during the requirements or design phase may only become apparent much later.
5. **Not suitable for large or dynamic projects:** Projects with frequently changing requirements are not a good fit for this model.

6 When to Use the Waterfall Model

The Waterfall Model is most suitable when:

- Requirements are clear, complete, and stable.
- The project is small or medium in size.
- Technology and tools are well understood.
- The customer does not expect frequent changes.
- Documentation and formal approval are important.

7 Why Does the Waterfall Model Fail for Software Projects?

The Waterfall Model is not a bad model, but it often fails in modern software projects because software requirements frequently change during development. The model assumes that all requirements are known from the beginning, which is rarely true in real-world projects.

In software development, customers may discover new needs after seeing early versions of a product, technologies may change, and market expectations may shift. Since the Waterfall Model does not easily support repeated feedback and revision, these changes can create delays, extra cost, and project risk.

7.1 Key Reasons for Failure

- **Requirements are difficult to change:** Once development begins, changing approved requirements can affect design, coding, testing, and delivery schedules.
- **Customer sees the product only at the end:** Users often cannot evaluate the working software until late in the project, so misunderstandings may remain hidden for a long time.

- **Testing happens late:** Defects are usually discovered after implementation, making them harder and more expensive to fix.
- **Mistakes are discovered late:** Errors in requirements or design may not become visible until the final stages of development.
- **Changes are expensive:** A small change requested late in the project may require major rework across multiple completed phases.
- **Long development cycle:** The sequential process can delay delivery of usable software.
- **Limited customer feedback:** Minimal customer involvement after requirements approval can lead to a final product that does not fully meet user expectations.
- **High project risk:** Late discovery of problems increases the risk of budget overruns, missed deadlines, or project failure.
- **Not suitable for dynamic projects:** Projects with changing requirements, evolving business goals, or uncertain features need a more flexible approach.
- **Poor adaptability:** The model does not respond well to new technologies, market changes, or competitive pressure during development.

8 What Is an Agile Methodology?

Agile methodology is a modern and flexible approach to software development. Instead of completing the entire project in one long sequence, Agile divides the project into small parts called iterations or sprints. Each sprint produces a small, working part of the software that can be reviewed, tested, and improved.

Agile focuses on customer collaboration, continuous feedback, quick delivery, and the ability to adapt to changing requirements. It is especially useful for software projects where requirements are not fully known at the beginning or may change during development.

8.1 How Agile Works

In Agile development, teams plan, design, code, test, and review small features in repeated cycles. After each cycle, the customer or user can provide feedback. The development team then uses that feedback to improve the product in the next cycle.

This process continues until the final software product is complete. Because Agile delivers working software frequently, problems can be identified early and changes can be handled more easily.

8.2 Key Characteristics of Agile Methodology

- It follows an iterative and incremental development process.
- Requirements can change during development.
- Customers are involved throughout the project.

- Working software is delivered frequently.
- Testing is performed continuously.
- Team communication and collaboration are highly important.
- It supports flexibility and quick adaptation to change.

8.3 Advantages of Agile Methodology

1. **Flexible to changes:** Agile allows teams to adjust requirements and priorities as the project progresses.
2. **Early delivery of software:** Users can see and test working features early instead of waiting until the end of the project.
3. **Continuous customer feedback:** Regular feedback helps ensure that the final product meets user needs.
4. **Early defect detection:** Continuous testing helps identify and fix errors sooner.
5. **Improved collaboration:** Developers, testers, customers, and stakeholders work closely together.
6. **Lower project risk:** Frequent review and delivery reduce the chances of major failure at the end.

8.4 Disadvantages of Agile Methodology

1. **Requires active customer involvement:** Agile works best when customers are available to give regular feedback.
2. **Less predictable planning:** Cost, time, and scope may be harder to estimate when requirements keep changing.
3. **Needs experienced team members:** Agile teams must communicate well and make quick decisions.
4. **Documentation may be limited:** Agile focuses more on working software, so documentation can sometimes be less detailed.
5. **Scope creep can occur:** Frequent changes may increase the project scope if not managed carefully.

8.5 Examples of Agile Frameworks

Common Agile frameworks include Scrum, Kanban, Extreme Programming (XP), Lean Software Development, and Crystal. These frameworks follow Agile principles but use different practices for planning, teamwork, and delivery.

9 Why Is Agile Popular in the Software Industry?

Agile is popular in the software industry because it matches the real nature of software development. In many software projects, requirements change, customers refine their ideas, technologies evolve, and competitors release new products. Agile helps teams respond to these changes quickly instead of following a rigid fixed plan.

Another major reason for Agile's popularity is that it delivers working software in small and frequent releases. This allows customers, users, and stakeholders to see progress early, provide feedback, and guide the product in the right direction. As a result, the final software is more likely to meet actual user needs.

9.1 Key Reasons for Agile's Popularity

- **Handles changing requirements:** Agile allows teams to update features and priorities as business needs change.
- **Delivers software faster:** Small releases help teams provide usable software earlier than traditional models.
- **Improves customer satisfaction:** Regular customer feedback ensures that the product stays aligned with user expectations.
- **Reduces project risk:** Frequent testing and review help detect problems early.
- **Supports teamwork:** Agile encourages close collaboration among developers, testers, customers, and project managers.
- **Improves product quality:** Continuous testing and improvement help produce more reliable software.
- **Adapts to market changes:** Teams can quickly respond to new technologies, customer demands, and business competition.
- **Increases transparency:** Regular meetings, sprint reviews, and progress tracking make the project status easier to understand.

Because of these benefits, Agile is widely used in startups, large companies, web development, mobile app development, cloud-based systems, and many other modern software projects.

10 Reasons for Setbacks of the Agile Model

Although Agile is popular and effective for many software projects, it can face setbacks when it is not implemented properly. Agile requires discipline, communication, customer involvement, and an experienced team. If these conditions are missing, Agile projects may become disorganized, costly, or difficult to control.

10.1 Common Reasons for Agile Setbacks

- **Lack of customer involvement:** Agile depends on regular customer feedback. If the customer is unavailable or unclear, the team may build features that do not meet user needs.

- **Unclear requirements:** Agile allows changing requirements, but too much uncertainty can confuse the team and delay development.
- **Poor communication:** Agile requires frequent communication among developers, testers, managers, and customers. Weak communication can lead to misunderstandings and rework.
- **Scope creep:** Continuous changes and new feature requests can increase the project scope if priorities are not controlled carefully.
- **Inexperienced team members:** Agile teams need self-management, technical skill, and quick decision-making. Lack of experience can reduce productivity.
- **Weak project planning:** Agile is flexible, but it still needs proper sprint planning, estimation, and tracking. Poor planning can cause missed deadlines.
- **Insufficient documentation:** Agile focuses on working software, so teams may produce less documentation. This can create problems during maintenance or knowledge transfer.
- **Difficulty in large projects:** Agile can be harder to manage when many teams, departments, or stakeholders are involved.
- **Frequent changes:** Regular changes can improve the product, but excessive changes may disrupt development and increase cost.
- **Lack of management support:** Agile needs support from leadership. Without proper support, teams may struggle with resources, decisions, and priorities.

10.2 Summary

The Agile Model can face setbacks when flexibility is not balanced with planning and control. To make Agile successful, teams must maintain clear communication, active customer participation, disciplined sprint planning, and continuous testing.

11 What Is DevOps?

DevOps is a modern software development and operations approach that combines development (Dev) and IT operations (Ops). Its main goal is to improve collaboration between software developers, testers, system administrators, and operations teams so that software can be built, tested, released, and maintained faster and more reliably.

In traditional software development, developers write code and then hand it over to the operations team for deployment and maintenance. This separation can cause delays, communication gaps, deployment errors, and slow problem resolution. DevOps removes these barriers by encouraging shared responsibility, automation, continuous feedback, and teamwork throughout the software life cycle.

11.1 Main Idea of DevOps

The main idea of DevOps is to deliver high-quality software quickly and continuously. DevOps teams use automation tools and collaborative practices to reduce manual work, detect errors early, and make software releases more frequent and reliable.

DevOps is not only a set of tools; it is also a culture and working style. It encourages teams to work together, share responsibility, monitor systems continuously, and improve the software delivery process over time.

11.2 DevOps Life Cycle

The DevOps life cycle includes several important stages:

1. **Planning:** Teams define requirements, project goals, tasks, and priorities.
2. **Development:** Developers write and update source code based on planned features.
3. **Build:** The source code is compiled and packaged into a deployable form.
4. **Testing:** Automated and manual tests are performed to detect bugs and quality issues.
5. **Release:** The tested software version is prepared for deployment.
6. **Deployment:** The software is delivered to production or user environments.
7. **Operation:** The application is managed, configured, and supported after deployment.
8. **Monitoring:** Teams continuously observe performance, errors, security issues, and user experience.

11.3 Key Practices of DevOps

- **Continuous Integration (CI):** Developers frequently merge code changes into a shared repository, where automated builds and tests check the quality of the code.
- **Continuous Delivery (CD):** Software is kept in a release-ready state so that it can be deployed quickly and safely.
- **Continuous Deployment:** Every successful change can be automatically deployed to production without manual approval.
- **Infrastructure as Code (IaC):** Servers, networks, and system configurations are managed using code instead of manual setup.
- **Automation:** Repetitive tasks such as testing, building, deployment, and configuration are automated to reduce errors and save time.
- **Monitoring and logging:** Applications and infrastructure are continuously monitored to detect failures, performance issues, and security risks.
- **Collaboration and communication:** Development and operations teams work together instead of functioning as separate groups.

11.4 Benefits of DevOps

1. **Faster software delivery:** Automation and continuous delivery help teams release software more quickly.

2. **Improved software quality:** Continuous testing helps detect bugs early and improves reliability.
3. **Better collaboration:** Developers, testers, and operations teams work together with shared goals.
4. **Reduced deployment failures:** Automated deployment reduces manual mistakes and makes releases more consistent.
5. **Quick problem recovery:** Monitoring and logging help teams identify and fix issues faster.
6. **Higher customer satisfaction:** Faster updates, fewer errors, and reliable systems improve the user experience.
7. **Efficient resource usage:** Automated infrastructure management helps teams use servers, cloud services, and tools more effectively.

11.5 DevOps Tools

DevOps uses many tools to support automation, collaboration, testing, deployment, and monitoring. Common examples include Git for version control, Jenkins and GitHub Actions for continuous integration, Docker for containerization, Kubernetes for container orchestration, Ansible and Terraform for infrastructure automation, and Prometheus or Grafana for monitoring.

11.6 Define the Term CI/CD in Relation with DevOps

CI/CD is one of the most important practices in DevOps. It helps software teams build, test, and release applications quickly, safely, and repeatedly through automation.

CI stands for **Continuous Integration**. It is the practice of frequently merging developers' code changes into a shared repository. After each code change, automated builds and tests are run to check whether the new code works correctly with the existing code. This helps teams find errors early and avoid integration problems.

CD can mean **Continuous Delivery** or **Continuous Deployment**. Continuous Delivery means that the software is always kept in a release-ready state, but the final release to production may still require manual approval. Continuous Deployment goes one step further by automatically deploying every successfully tested change to production.

11.6.1 Importance of CI/CD in DevOps

- **Faster releases:** CI/CD automates building, testing, and deployment, so software can be released more quickly.
- **Early bug detection:** Automated tests run after every change and help identify defects early.
- **Reduced manual work:** Repetitive tasks such as builds and deployments are handled automatically.
- **Improved software quality:** Frequent testing and validation make the software more stable and reliable.

- **Better collaboration:** Developers, testers, and operations teams can work together using a shared pipeline.
- **Lower deployment risk:** Smaller and frequent releases are easier to test, monitor, and roll back if needed.

11.6.2 Common CI/CD Tools

Common CI/CD tools include Jenkins, GitHub Actions, GitLab CI/CD, Azure DevOps, CircleCI, Travis CI, Bamboo, and TeamCity. These tools help automate code integration, testing, packaging, and deployment.

11.6.3 Basic CI/CD Workflow

A typical CI/CD workflow follows these steps:

1. A developer writes code and commits it to a shared version control repository.
2. The CI/CD tool automatically detects the new code change.
3. The application is built and compiled automatically.
4. Automated tests are executed to check whether the code is correct and stable.
5. If the tests pass, the application is packaged as a deployable artifact.
6. The artifact is delivered or deployed to a testing, staging, or production environment.
7. Monitoring tools track the application after deployment and provide feedback to the team.

11.6.4 Example of CI/CD in DevOps

For example, when a developer pushes code to GitHub, GitHub Actions or Jenkins can automatically start a pipeline. The pipeline may compile the code, run unit tests, scan for code quality issues, create a Docker image, and deploy the application to Kubernetes. If any step fails, the team receives feedback immediately and can fix the problem before it reaches users.

11.6.5 Difference Between CI, Continuous Delivery, and Continuous Deployment

- **Continuous Integration:** Focuses on merging and testing code changes frequently.
- **Continuous Delivery:** Keeps the software ready for release, but deployment to production is usually approved manually.
- **Continuous Deployment:** Automatically releases every successful change to production without manual approval.

11.6.6 Summary of CI/CD

CI/CD supports the DevOps goal of fast, reliable, and automated software delivery. It reduces delays, improves quality, increases team productivity, and helps organizations release updates to users more frequently.

11.7 Define the Term IaC

IaC stands for **Infrastructure as Code**. It is a DevOps practice in which infrastructure such as servers, networks, databases, storage, and configuration settings are created and managed using code instead of manual setup.

In traditional infrastructure management, system administrators manually configure servers and environments. This can be slow, error-prone, and difficult to repeat. With IaC, infrastructure is written in configuration files or scripts, stored in version control, reviewed by teams, and executed automatically by tools.

11.7.1 How IaC Works

In IaC, the required infrastructure is described in code. The IaC tool reads this code and automatically creates or updates the required environment. For example, a team can write code to create cloud servers, install software, configure networks, and deploy applications.

11.7.2 Benefits of IaC

- **Consistency:** The same code can create the same environment every time.
- **Speed:** Infrastructure can be created or updated quickly through automation.
- **Reduced human error:** Automated setup reduces mistakes caused by manual configuration.
- **Version control:** Infrastructure changes can be tracked, reviewed, and restored if needed.
- **Scalability:** Teams can easily create multiple servers or environments using the same code.
- **Better collaboration:** Developers and operations teams can work together on infrastructure files.

11.7.3 Common IaC Tools

Common Infrastructure as Code tools include Terraform, Ansible, AWS CloudFormation, Azure Resource Manager, Pulumi, and Chef. These tools help teams automate infrastructure creation, configuration, and management.

11.8 DevOps Compared with Agile

Agile focuses mainly on improving the software development process through iterative planning, feedback, and frequent delivery of features. DevOps extends this idea by also improving deployment, operations, monitoring, and maintenance. In simple terms, Agile

helps teams build software better, while DevOps helps teams deliver and operate software better.

11.9 Challenges of DevOps

- It requires a cultural change in how teams work and communicate.
- Team members need technical skills in automation, cloud platforms, testing, and monitoring.
- Setting up DevOps tools and pipelines can be complex at the beginning.
- Poorly managed automation can create security or reliability problems.
- Organizations may face resistance when changing from traditional methods to DevOps practices.

11.10 Conclusion on DevOps

DevOps is an important approach in modern software engineering because it connects development and operations into one collaborative process. It helps organizations deliver software faster, improve quality, reduce failures, and respond quickly to user needs and system problems.

12 What Problems Does DevOps Solve in the Software Industry?

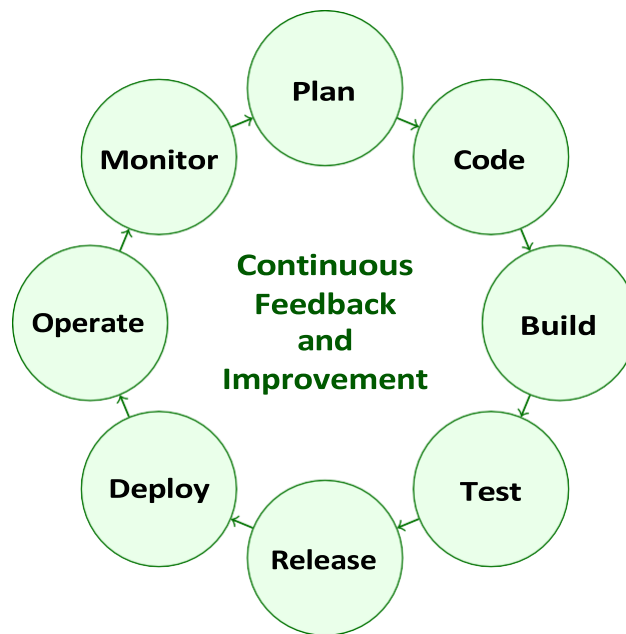
DevOps solves many common problems found in traditional software development and IT operations. In older approaches, development teams, testing teams, and operations teams often worked separately. This separation caused slow releases, deployment failures, poor communication, and delays in fixing production issues. DevOps solves these problems by using automation, continuous testing, shared responsibility, monitoring, and close collaboration.

12.1 Problems Solved by DevOps

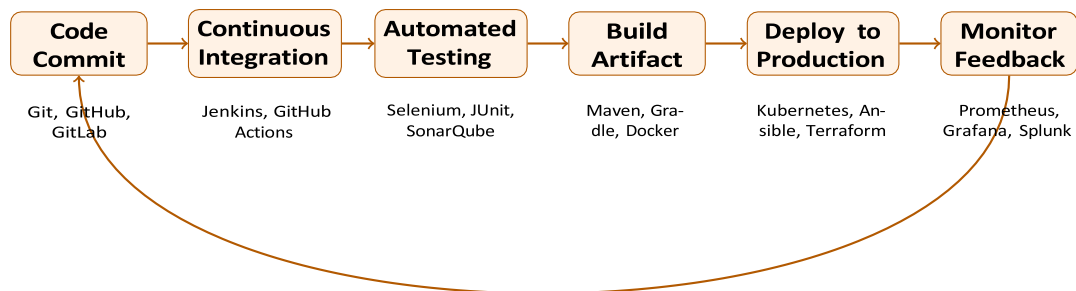
- **Slow software delivery:** DevOps uses automation and continuous delivery to release software faster.
- **Poor communication between teams:** DevOps encourages developers, testers, and operations teams to work together with shared goals.
- **Manual deployment errors:** Automated deployment reduces human mistakes and makes releases more reliable.
- **Late bug detection:** Continuous integration and automated testing help detect bugs early in the development process.
- **Unstable production systems:** Monitoring and logging help teams identify failures, performance issues, and security risks quickly.

- **Long recovery time:** DevOps improves incident response by using alerts, logs, dashboards, and rollback strategies.
- **Inconsistent environments:** Infrastructure as Code helps create consistent development, testing, and production environments.
- **Low customer satisfaction:** Faster updates and more reliable applications help meet customer needs more effectively.
- **Lack of visibility:** DevOps tools provide dashboards, reports, logs, and metrics that make project and system status easier to understand.

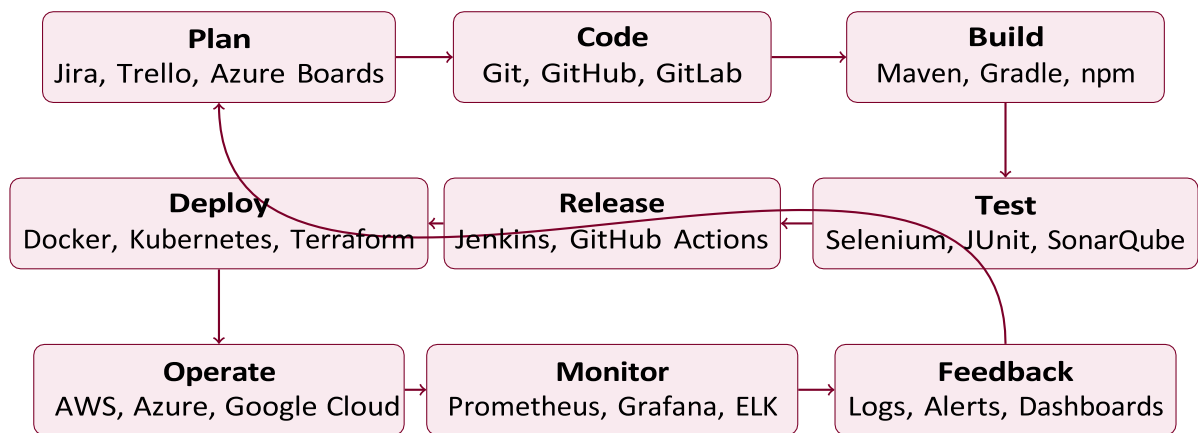
12.2 DevOps Life Cycle Diagram



12.3 CI/CD Pipeline Diagram



12.4 DevOps Toolchain Overview Diagram



12.5 DevOps Phases and Common Tools

DevOps Phase	Purpose	Common Tools
Plan	Define requirements, tasks, sprint goals, and project priorities.	Jira, Trello, Azure Boards, Confluence, Asana
Code	Write, review, and manage source code collaboratively.	Git, GitHub, GitLab, Bitbucket, Visual Studio Code
Build	Compile code, package applications, and create build artifacts.	Maven, Gradle, npm, Ant, MS-Build
Test	Run automated tests to check quality, security, and functionality.	Selenium, JUnit, TestNG, PyTest, Postman, SonarQube
Release	Prepare approved builds for production deployment.	Jenkins, GitHub Actions, GitLab CI/CD, Azure DevOps, Bamboo
Deploy	Deploy applications to servers, containers, or cloud platforms.	Docker, Kubernetes, Ansible, Terraform, Helm, Argo CD
Operate	Manage application infrastructure, configuration, and production services.	Kubernetes, Docker, AWS, Microsoft Azure, Google Cloud, OpenShift
Monitor	Track performance, logs, errors, availability, and user experience.	Prometheus, Grafana, ELK Stack, Splunk, Nagios, Datadog

12.6 Summary

DevOps solves software industry problems by making development, testing, deployment, operations, and monitoring more connected and automated. It reduces delays, improves quality, increases reliability, and helps organizations deliver better software to users faster.

13 Conclusion

The Waterfall Model is a traditional SDLC model that follows a simple, sequential process. It is easy to understand and manage, making it useful for projects with fixed requirements. However, because changes are difficult and testing occurs late, it is less suitable for complex projects where requirements may change frequently. Agile methodology provides a more flexible alternative by supporting continuous feedback, frequent delivery, and adaptation to changing software requirements. DevOps further improves modern software delivery by connecting development and operations through collaboration, automation, continuous testing, deployment, and monitoring.